

专题：筛选素数

如何筛选N以内的素数？

试除法

- 每一个数都去判断是否有除了1和本身以外的因数。
- 时间复杂度 $O(n^2)$

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int isprime(int n) {
```

```
    for (int i = 2; i < n ; i++) {
```

```
        if (n % i == 0) {
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

$i \leq n/2$

$i \leq \text{sqrt}(n)$

埃氏筛(埃拉托斯特尼筛法)

- 一个素数的倍数一定是合数。
- 埃氏筛法的基本原理是从最小的素数2开始，将序列中所有2的倍数（除了2本身）标记为非素数，然后找到剩下的数中最小的数，它一定是素数，接着将这个素数的所有倍数（除了该素数本身）标记为非素数。这个过程一直重复，直到处理完所有小于或等于给定整数n的数。
- $O(N \log \log N)$

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30

```
vector<int> primes;
```

```
void sieve_Eratosthenes(int n) {  
    vector<bool> is_prime(n + 1, true);  
    for (int i = 2; i <= n; i++) { 如果用i<=sqrt(n), 后面要补录  
        if (is_prime[i]) {  
            primes.push_back(i);  
            for (int j = i * 2; j <= n; j += i) { int j = i * i  
                is_prime[j] = false;  
            }  
        }  
    }  
}
```

线性筛（欧拉筛法）

- 线性筛法的核心原理：每个合数必有一个最小质因子 P ，用这个因子 $P * M$ 筛掉合数 N 。每个合数只被它的最小质因子筛掉一次，从而避免了重复筛选。
- 时间复杂度： $O(n)$

编写程序

- 枚举 $2 \sim n$ 中的每一个数
 - 如果 i 是素数则保存到素数表中
 - 利用 i 和素数表中的素数 $\text{prime}[j]$ 去筛除 $i * \text{prime}[j]$ ，为了确保 $i * \text{prime}[j]$ 只被筛一次，要确保 $\text{prime}[j]$ 是 $i * \text{prime}[j]$ 中的最小素因子，即 i 中不能有比 $\text{prime}[j]$ 还要小的素因子。
 - $i * \text{prime}[j](2, 3, 5, 7, \dots)$

	素数表	筛除的数		素数表	筛除的数
<u>2</u>	{2}	{4}	<u>13</u>	{2, 3, 5, 7, 11, 13}	{26, 39}
<u>3</u>	{2, 3}	{6, 9}	<u>14</u>	{2, 3, 5, 7, 11, 13}	{28}
<u>4</u>	{2, 3}	{8}	<u>15</u>	{2, 3, 5, 7, 11, 13}	{30, 45}
<u>5</u>	{2, 3, 5}	{10, 15, 25}	<u>16</u>	{2, 3, 5, 7, 11, 13}	{32}
<u>6</u>	{2, 3, 5}	{12}	<u>17</u>	{2, 3, 5, 7, 11, 13, 17}	{34}
<u>7</u>	{2, 3, 5, 7}	{14, 21, 35, 49}	<u>18</u>	{2, 3, 5, 7, 11, 13, 17}	{36}
<u>8</u>	{2, 3, 5, 7}	{16}	<u>19</u>	{2, 3, 5, 7, 11, 13, 17, 19}	{38}
<u>9</u>	{2, 3, 5, 7}	{18, 27}	<u>20</u>	{2, 3, 5, 7, 11, 13, 17, 19}	{20}
<u>10</u>	{2, 3, 5, 7}	{20}	<u>21</u>	{2, 3, 5, 7, 11, 13, 17, 19}	{42}
<u>11</u>	{2, 3, 5, 7, 11}	{22, 33}	<u>22</u>	{2, 3, 5, 7, 11, 13, 17, 19}	{44}
<u>12</u>	{2, 3, 5, 7, 11}	{24}	...	...	...

i能够整除素数表中的某个数时，就不往下了，因为i里面含有一个更小的质因数。

```
vector<int>primes;
int cnt=0;
void Prime(int n){
    vector<bool>is_prime(n+1,true);
    for (int i = 2;i <= n; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            cnt++;}
        for (int j = 0; j <=cnt && i*primes[j] <= n; j++) {
            is_prime[i*primes[j]] = false;
            if (i % primes[j] == 0) break;
        }
    }
}
```

线性筛一定比埃氏筛快？

在 C++ 中，埃氏筛（埃拉托斯特尼筛法）和线性欧拉筛（欧拉筛）的效率对比并非绝对，而是取决于具体场景，但通常情况下线性欧拉筛更优，尤其在处理大规模数据时优势更明显。

对于小规模数据 ($n < 1e6$) 或简单场景，埃氏筛因其实现简单，效率反而更高。

1. 指令数量更少，执行开销更低
2. 分支更少，减少 CPU 流水线阻塞
3. 数据访问模式更简单，缓存利用率更高
4. 编译器优化更友好